

Aula 8 — Expansão e regularização

Expansão de atributos; Ridge (L2); Lasso (L1); Elastic Net

Prof. Dr. João Paulo Ferreira de Mello

18 de setembro de 2026

Ciências de Dados

Fatec Rubens Lara – Baixada Santista

O que vamos ver hoje

- **De onde viemos:** a Aula 7 nos deu como *medir* o excesso de flexibilidade — a validação cruzada. Medir, porém, não é consertar.
- **O novo:** dois movimentos opostos. A **expansão** fabrica preditores novos (o acelerador); a **regularização** penaliza coeficientes grandes (o freio). Ridge, Lasso e Elastic Net são o freio.
- **Para onde vai:** é a última peça do kit supervisionado — e o último conteúdo antes da **P1**.

O fio da aula: a imobiliária agora tem **20 características** por imóvel — e apenas **35 imóveis** para aprender com elas.

Quando sobram preditores

Os dados de hoje: 20 características, 35 imóveis

```
dim(dados)           # 35 imoveis, 1 resposta + 20 preditores
```

```
#> [1] 35 21
```

```
names(dados)[1:8]    # os oito primeiros nomes
```

```
#> [1] "preco"      "area"        "area_util"   "quartos"
```

```
#> [5] "idade"      "dist_centro" "banheiros"   "vagas"
```

Como os dados são *simulados*, sabemos a verdade — e o modelo não:

$$\text{preço} = 120 + 1,6 \text{ área} + 25 \text{ quartos} - 2,2 \text{ idade} - 6 \text{ dist_centro} + \varepsilon, \quad \varepsilon \sim N(0, 35^2).$$

- **4 preditores** entram de verdade; **15** são ruído puro;
- e *area_util* é uma *cópia quase exata* de *area* — o problema da próxima página.

Lembrete: o MQO, nosso ponto de partida

Antes de criticá-lo, vale lembrar quem é o réu de hoje.

Definição

MQO — *mínimos quadrados ordinários* (em inglês *OLS*, *ordinary least squares*) — é o estimador da **Aula 4**: escolhe os coeficientes que minimizam a soma dos quadrados dos resíduos,

$$\hat{\beta}^{\text{MQO}} = \arg \min_{\beta} \underbrace{\sum_{i=1}^n (y_i - \hat{y}_i)^2}_{\text{RSS}}, \quad \text{com solução} \quad \hat{\beta}^{\text{MQO}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

- É exatamente o que o `lm()` calcula — toda regressão das Aulas 4, 6 e 7 foi MQO.
- “Ordinários” distingue dos parentes: *ponderados* (WLS), *generalizados* (GLS). Neste curso, MQO é o padrão.
- **Repare no que não há nessa fórmula:** nenhuma restrição sobre o *tamanho* dos $\hat{\beta}$. Ele erra o menos possível no treino, custe o que custar aos coeficientes — e é essa liberdade que a aula de hoje vai limitar.

O MQO com 20 preditores: ótimo no papel

```
m_mqo <- lm(preco ~ ., data = dados)
c(R2 = summary(m_mqo)$r.squared,
  R2_ajustado = summary(m_mqo)$adj.r.squared,
  RMSE_treino = rmse(fitted(m_mqo), yy),
  RMSE_teste = rmse(predict(m_mqo, novos), novos$preco))
```

```
#>           R2 R2_ajustado RMSE_treino RMSE_teste
#>      0.9210      0.8082      28.1928      75.5995
```

- $R^2 = 0,92$ e erro de treino de **28 mil reais**: o relatório fica lindo.
- Em imóveis novos, o erro é de **76 mil** — quase três vezes pior.
- Referência: um MQO com *só os 4 preditores verdadeiros* erra **38 mil**, e o piso irreduzível é **35 mil**.

Observação

Com $p = 20$ e $n = 35$, sobra liberdade para o modelo “explicar” o ruído. É o sobreajuste da Aula 6 — agora vindo do *número de preditores*, não do grau do polinômio.

Uma das doenças: preditores que dizem a mesma coisa

```
cor(dados$area, dados$area_util)           # quase 1

#> [1] 0.9883

summary(lm(preco ~ area, data = dados))$coef[2, 1:2]

#>      Estimate Std. Error
#>      1.568      0.219

summary(lm(preco ~ area + area_util,
            data = dados))$coef[2:3, 1:2]

#>      Estimate Std. Error
#> area      1.60368      1.456
#> area_util -0.04186      1.698
```

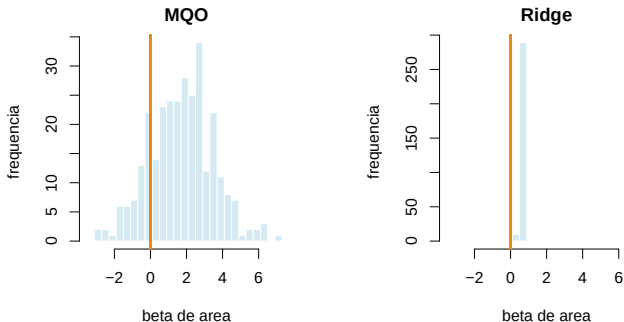
Leia a saída: sozinha, area tem erro padrão 0,22. Ao lado da gêmea, o coeficiente quase não muda (1,60) mas o erro padrão salta para 1,46 — **6,7 vezes maior**, e nenhuma das duas fica “significativa”. **Colinearidade:** com duas colunas dizendo a mesma coisa, os dados não sabem *a qual* atribuir o efeito — e o repartem de qualquer jeito. O ajuste é bom; os coeficientes, não.

Instável quanto? Reamostragem e veja (Aula 7)

Fixamos o par colinear e refazemos o ajuste em **300 reamostras bootstrap** — duas vezes: com MQO e com Ridge. A pergunta: o coeficiente de area fica onde?

```
set.seed(7)
est <- replicate(300, {                                # 300 reamostras bootstrap
  i <- sample(35, 35, replace = TRUE); d <- dados[i, ]
  Xp <- as.matrix(d[, c("area", "area_util")])
  c(coef(lm(preco ~ area + area_util, data = d))[2],      # MQO
    as.vector(coef(glmnet(Xp, d$preco, alpha = 0, lambda = 40)))[2]) # Ridge
})
par(mfrow = c(1,2), mar = c(4,4,2.2,1), pty = "s")
br <- seq(-3.5, 7.5, by = 0.4)
for (j in 1:2) {
  hist(est[j, ], breaks = br, xlim = c(-3.5, 7.5), col = cfill,
        border = "white", main = c("MQO", "Ridge")[j],
        xlab = "beta de area", ylab = "frequencia")
  abline(v = 0, col = "red", lwd = 3)
}
```

O mesmo coeficiente, 300 vezes



Leia a figura: o MQO varia de $-2,8$ a $6,9$, e em **53 das 300** reamostras o coeficiente de area sai *negativo* (“imóvel maior, mais barato”). O Ridge fica entre $0,44$ e $0,91$: desvio **20 vezes menor**. As escalas verticais diferem — o que importa é a largura. **Mas cuidado:** o Ridge não está *mais perto da verdade* — ele reparte o efeito entre as gêmeas e encolhe as duas. O que ele compra é **estabilidade**, ao preço de viés: a troca da Aula 6, feita de propósito.

Expansão de atributos

Antes do freio, o acelerador: expandir

Expansão de atributos é fabricar colunas novas a partir das que já existem — foi o que fizemos na Aula 6 ao criar area^2 e area^3 . As formas mais comuns, todas em uma fórmula do R:

```
lm(preco ~ poly(area, 3), data = dados)      # polinomio (Aula 6)
lm(preco ~ area + I(area^2), data = dados)    # potencia explicita
lm(preco ~ area * quartos, data = dados)      # interacao: area, quartos, area:quartos
lm(preco ~ log(area), data = dados)           # transformacao
lm(preco ~ factor(quartos), data = dados)     # qualitativa -> colunas 0/1
X <- model.matrix(preco ~ . * ., data = dados) # TODAS as interacoes de 2 a 2
dim(X)                                         # quantas colunas isso cria?
```

Observação

Expansão é **barata de escrever e cara de pagar**: $. * .$ com 20 preditores gera $20 + \binom{20}{2} = 210$ colunas — para $n = 35$ imóveis. A flexibilidade que resolve o viés cria a variância. **Daí o freio.**

Ridge (L2)

Ridge: pagar um pedágio por coeficiente grande

O MQO escolhe β minimizando só o erro. O **Ridge** acrescenta uma *multa* pelo tamanho dos coeficientes:

$$\underbrace{\sum_{i=1}^n (y_i - \beta_0 - \sum_j \beta_j x_{ij})^2}_{\text{RSS (o de sempre)}} + \lambda \underbrace{\sum_{j=1}^p \beta_j^2}_{\text{penalidade } L_2}.$$

Em palavras: o modelo ainda quer acertar os preços, mas agora cada coeficiente grande *custa*. Ele só “gasta” um β_j grande se aquele preditor pagar o próprio custo em redução de erro.

- $\lambda = 0$: nada muda — é o MQO.
- $\lambda \rightarrow \infty$: todos os $\beta_j \rightarrow 0$ — sobra a média.
- Entre os dois extremos mora o modelo útil. λ **é o novo botão de flexibilidade** — e quem o escolhe é a validação cruzada.

O intercepto β_0 *não* é penalizado: ele só posiciona a resposta, não mede efeito de preditor.

A conta fecha — e por que λ conserta a matriz

A solução do Ridge tem fórmula explícita, quase a da Aula 4:

$$\hat{\beta}^{\text{ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}.$$

```
Xs <- scale(X); yc <- yy - mean(yy)      # padronizar e centrar
for (lam in c(0.01, 50, 500)) {
  b <- solve(crossprod(Xs) + lam*diag(20), crossprod(Xs, yc))
  cat("lambda", format(lam, width = 6),
      "| soma |beta|:", format(round(sum(abs(b)), 1), width = 6),
      "| maior |beta|:", round(max(abs(b)), 1), "\n")
}
```

```
#> lambda    0.01 | soma |beta|:   368.5 | maior |beta|: 99.1
#> lambda      50 | soma |beta|:    115 | maior |beta|: 21.6
#> lambda     500 | soma |beta|:     25.5 | maior |beta|:  4.6
```

- Com colinearidade, $\mathbf{X}^T \mathbf{X}$ é quase singular e sua inversa explode — é isso que produz os coeficientes instáveis. Somar $\lambda \mathbf{I}$ engorda a diagonal e devolve uma matriz saudável.
- **Padronizar é obrigatório:** a penalidade $\sum \beta_j^2$ compara coeficientes entre si. Sem padronizar, trocar a área de m² para hectares mudaria o modelo escolhido.

No R: glmnet e a escolha de λ

```
library(glmnet)                                # glmnet quer MATRIZ, nao formula
set.seed(1)
cvr <- cv.glmnet(X, yy, alpha = 0)              # alpha = 0 <=> Ridge
c(lambda_min = cvr$lambda.min,
  RMSE_teste = rmse(predict(cvr, Xn, s = "lambda.min"), novos$preco))

#> lambda_min RMSE_teste
#>      21.78      52.66
```

- cv.glmnet percorre uma grade de λ e roda uma **validação cruzada** em cada um — é a Aula 7 inteira, embutida numa função.
- O erro de teste caiu de **76 mil** (MQO) para **53 mil**.
- glmnet já padroniza por dentro (standardize = TRUE) e devolve os coeficientes na escala original.

Observação

O Ridge encolhe *todos* os coeficientes, mas não zera nenhum: os 15 preditores de ruído continuam no modelo, cada um com sua migalha. Guarde isso — é exatamente o que o Lasso vai mudar.

Lasso (L1)

Lasso: trocar o quadrado pelo módulo

Uma única letra muda na fórmula:

$$\text{RSS} + \lambda \sum_{j=1}^p |\beta_j| \quad \text{em vez de} \quad \text{RSS} + \lambda \sum_{j=1}^p \beta_j^2.$$

E a consequência é enorme: o Lasso **zera** coeficientes.

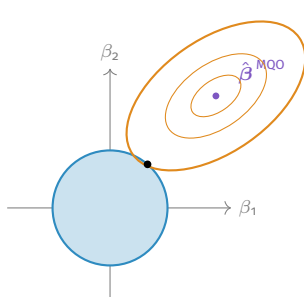
Por quê? Olhe o custo de reduzir um coeficiente pequeno, digamos de 0,02 para 0:

- no **Ridge**, a multa cai de $0,02^2 = 0,0004$ para 0 — um ganho *ridículo*, que nunca compensa. O coeficiente encolhe, mas sobrevive;
- no **Lasso**, a multa cai de 0,02 para 0 — proporcional ao que se remove. *Sempre* compensa, se o preditor não estiver pagando o próprio custo.

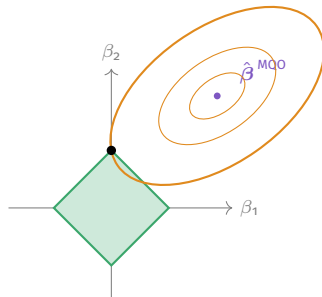
Em palavras: o módulo continua “cobrando caro” perto de zero; o quadrado desiste de cobrar. Por isso o Lasso faz **seleção de preditores** — e entrega, de brinde, um modelo que cabe numa frase.

A mesma história, em desenho

Penalizar equivale a *limitar* β a uma região em torno da origem. A solução é o ponto onde as elipses do RSS (centradas em $\hat{\beta}^{MQO}$) tocam essa região pela primeira vez:



Ridge: região *redonda*
toca fora dos eixos: $\beta_1, \beta_2 \neq 0$

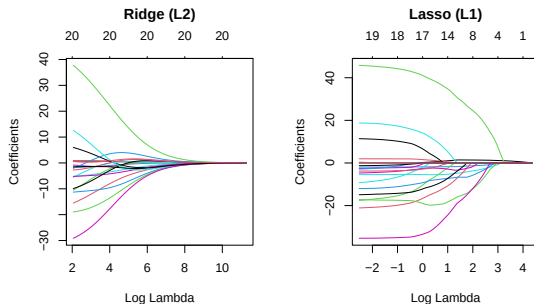


Lasso: região com *quinas*
toca numa quina: $\beta_1 = 0$

As quinas do losango ficam *sobre os eixos* — e é lá que uma elipse que vem de longe encosta primeiro. Em p dimensões há quinas, arestas e faces: daí o Lasso zerar vários de uma vez.

Os dois caminhos, lado a lado

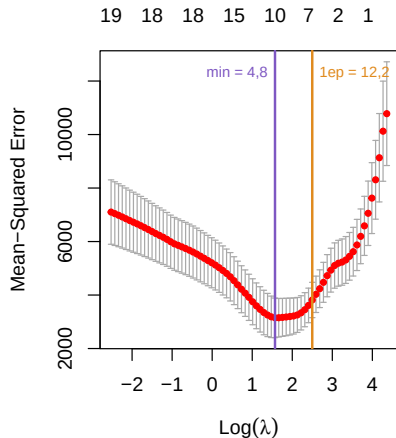
```
gr <- glmnet(X, yy, alpha = 0)           # Ridge
gl <- glmnet(X, yy, alpha = 1)           # Lasso
par(mfrow = c(1,2), mar = c(4,4,3.6,1), pty = "s")
plot(gr, xvar = "lambda"); title("Ridge (L2)", line = 2.6)
plot(gl, xvar = "lambda"); title("Lasso (L1)", line = 2.6)
```



Leia a figura: no **Ridge** as curvas se *aproximam* de zero sem chegar; no **Lasso** *encostam* e ficam — e o eixo de cima conta os sobreviventes.

Escolhendo λ : é a Aula 7 outra vez

```
set.seed(1)
cvl <- cv.glmnet(X, yy, alpha = 1) # Lasso
par(mar = c(4,4,3.4,1), pty = "s")
plot(cvl) # o eixo x sai em LOG de lambda
# ... então anotamos o lambda de verdade:
rot <- c("min = 4,8", "1ep = 12,2")
dx <- c(-0.18, 0.18); al <- c(1, 0)
for (k in 1:2) {
  L <- c(cvl$lambda.min, cvl$lambda.1se)[k]
  abline(v = log(L), lwd = 2,
        col = c(croxo, claranja)[k])
  text(log(L) + dx[k], par("usr")[4] - 800,
       rot[k], col = c(croxo, claranja)[k],
       adj = c(al[k], 0.5), cex = 0.72)
}
```



Os três eixos: embaixo, $\log \lambda$ — não λ , daí a anotação; à esquerda, o MSE de validação cruzada com barras de ± 1 erro padrão; em cima, quantos preditores sobrevivem a cada λ .

Os dois candidatos, em números

```
i1 <- which(cvl$lambda == cvl$lambda.min); i2 <- which(cvl$lambda == cvl$lambda.1se)
data.frame(lambda = round(cvl$lambda[c(i1,i2)], 2), CV_mse = round(cvl$cvm[c(i1,i2)]),
           CV_rmse = round(sqrt(cvl$cvm[c(i1,i2)]), 1),
           preditores = as.integer(cvl$nzzero[c(i1,i2)]),
           row.names = c("lambda.min", "lambda.1se"))
```

```
#>           lambda CV_mse CV_rmse preditores
#> lambda.min    4.80   3148    56.1         10
#> lambda.1se   12.17   3815    61.8          7
```

A regra de **um erro padrão** — que define o segundo candidato — é esta conta:

```
c(CV_min = round(cvl$cvm[i1]),          ep = round(cvl$cvsd[i1]),
  limiar = round(cvl$cvm[i1] + cvl$cvsd[i1]), CV_1se = round(cvl$cvm[i2]))
```

```
#> CV_min    ep limiar CV_1se
#>   3148    741   3889   3815
```

$3\,815 \leq 3\,889$: o erro de `lambda.1se` cabe dentro de *uma* barra do mínimo — trocamos 3 preditores (10 → 7) por uma perda que a CV não distingue de ruído. **E o placar adiante, que diz 46?** A CV ajusta cada dobra com menos imóveis que o modelo final: é **pessimista** (Aula 7) — e 741 sobre 3 148 é $\pm 24\%$.

Quem o Lasso guarda — e o que ele erra

```
cl <- coef(cvl, s = "lambda.1se")  
round(cl[as.vector(cl) != 0, ], 2)           # so os sobreviventes
```

```
#> (Intercept)      area      quartos      idade dist_centro  
#>      136.94      1.31      18.04      -0.99      -2.52  
#> condominio      ruído      onibus  
#>      -2.93      -6.12      -3.88
```

- **Acertou:** os quatro verdadeiros — area, quartos, idade, dist_centro — estão todos lá, com os sinais certos.
- **Errou:** condominio, ruído e onibus são ruído puro e entraram assim mesmo. **Seleção não é prova de causa.**
- **Descartou area_util:** entre duas colunas gêmeas, o Lasso escolhe *uma* e zera a outra — a escolha é meio arbitrária.
- Os coeficientes saem *encolhidos* (1,31 contra a verdade 1,60): é o viés que se aceita em troca de variância (Aula 6).

Comparar e decidir

O placar desta base

```
set.seed(1); cve <- cv.glmnet(X, yy, alpha = 0.5) # Elastic Net
round(c(
  MQO_20      = rmse(predict(m_mqo, novos), novos$preco),
  Ridge       = rmse(predict(cvr, Xn, s = "lambda.min"), novos$preco),
  Lasso_1se   = rmse(predict(cvl, Xn, s = "lambda.1se"), novos$preco),
  ElasticNet  = rmse(predict(cve, Xn, s = "lambda.1se"), novos$preco),
  MQO_4_certos = rmse(predict(lm(preco ~ area + quartos + idade +
                                dist_centro, data = dados), novos),
                                novos$preco)), 1)

#>      MQO_20      Ridge      Lasso_1se      ElasticNet
#>      75.6      52.7      45.9      48.7
#> MQO_4_certos
#>      38.4
```

RMSE em mil R\$; o piso irreduzível é 35. O Lasso chegou a **46** com 7 preditores — perto dos **38** de quem conhecia a resposta, e muito à frente dos **76** do MQO com tudo.

Observação

O Lasso ganhou aqui porque a verdade é esparsa: 4 preditores de 20. Se fosse “muitos efeitos pequenos”, o Ridge ganharia — e você não sabe de antemão qual é o caso. Por isso se testam os dois, por validação cruzada.

Elastic Net: os dois pedágios ao mesmo tempo

$$\text{RSS} + \lambda \left[(1 - \alpha) \sum_j \beta_j^2 + \alpha \sum_j |\beta_j| \right], \quad 0 \leq \alpha \leq 1.$$

Agora há **dois** botões: λ diz *quanto* penalizar; α diz *de que jeito*. E $\alpha = 0$ é Ridge, $\alpha = 1$ é Lasso.

Por que ele existe? Volte ao par `area/area_util`. O Lasso escolhe uma e joga a outra fora — e, numa amostra diferente, pode escolher a outra. O Ridge fica com as duas, dividindo o efeito. O Elastic Net faz o meio-termo: **agrupa** preditores correlacionados, mantendo ou descartando o grupo inteiro.

Observação

Regra prática: preditores muito correlacionados (sensores, pixels, colunas derivadas umas das outras) $\Rightarrow$ Elastic Net. No `glmnet`, basta `alpha = 0.5` — e α também pode ser escolhido por validação cruzada.

Exercícios e trabalho

Exercício

Um modelo tem $\beta = (3, -4, 0,5)$ (sem contar β_0). Calcule a penalidade L_2 e a penalidade L_1 . Se $\lambda = 2$, quanto cada uma acrescenta ao RSS?

Exercício

Com λ crescendo de 0 a ∞ , diga para onde vão: (a) os coeficientes; (b) o erro de *treino*; (c) o viés; (d) a variância.

Exercício

Uma validação cruzada deu, para o Lasso: $\lambda = 0,5 \Rightarrow CV = 2\,700$ (14 preditores); $\lambda = 2 \Rightarrow CV = 2\,050$ (9); $\lambda = 6 \Rightarrow CV = 2\,120$ (5); $\lambda = 20 \Rightarrow CV = 3\,400$ (1). Qual é o $\lambda_{\min}$? Se o erro padrão no fundo do U é 120, qual λ a regra de *um erro padrão* escolheria, e com quantos preditores?

Exercício (Por que padronizar)

area está em m^2 e dist_centro em km. Explique por que aplicar Ridge sem padronizar dá um modelo diferente de aplicá-lo com as áreas em hectares. Por que o MQO *não* tem esse problema?

Exercício (Zera ou não zera)

Explique, comparando o custo de levar um coeficiente de 0,02 a 0, por que o Lasso zera e o Ridge não. Relacione com o desenho do losango e do círculo.

Exercício (Seleção não é causa)

O Lasso guardou ruído, que por construção não afeta o preço. Como isso é possível, e o que você escreveria no relatório sobre essa variável?

Exercício (Qual usar)

Diga qual método você tentaria primeiro: (a) 500 preditores, suspeita de que uns 10 importam; (b) 30 preditores, todos plausíveis, vários correlacionados; (c) 3 preditores e $n = 5000$.

```
library(glmnet)
X <- model.matrix(y ~ ., data = dados)[, -1] # tira a coluna do intercepto
yv <- dados$y

set.seed(1)
cvl <- cv.glmnet(X, yv, alpha = 1)           # Lasso
cvr <- cv.glmnet(X, yv, alpha = 0)           # Ridge
plot(cvl)                                    # a curva em U

cl <- coef(cvl, s = "lambda.1se")
cl[as.vector(cl) != 0, ]                     # quem sobreviveu
c(lasso = min(cvl$cvm), ridge = min(cvr$cvm)) # erro de CV de cada um
```

Entregar (código em estrutura/codigo/, registro em consolidados/): a curva em U, o λ escolhido e por quê, a lista dos preditores que sobreviveram, e *uma frase* dizendo o que o Lasso disse sobre o seu problema — quais variáveis ele considera que carregam o sinal.

- **Expansão** fabrica preditores e compra flexibilidade; **regularização** é o freio.
- **Ridge** (L_2) encolhe tudo e nada zero — cura colinearidade e estabiliza coeficientes.
- **Lasso** (L_1) zera — faz seleção de preditores, e entrega um modelo curto.
- λ (e α) saem da **validação cruzada**; **padronizar** é obrigatório.

Próxima aula: revisão dos Blocos I–II e **P1**. Depois dela, viramos a chave: modelos que não supõem forma fixa — suavização, *splines* e GAM.